Using Quantum-Resistant Key Encapsulation Mechanisms (KEMs) in the Pairwise Mode of Group Object Security for Constrained RESTful Environments (Group OSCORE)

draft-tiloca-core-group-oscore-kem-00

Marco Tiloca, RISE

IETF 126 meeting – Vienna – July 23rd, 2026

Motivation

- **Group OSCORE [1] is used to protect group communication for CoAP**
- **Is Group OSCORE quantum-resistant?**
- **Group mode**
  - A signature is computed by the message sender
  - One can use a quantum-resistant Signature Algorithm, e.g., ML-DSA (FIPS 204)
- **Pairwise mode**
  - Pairwise symmetric keys are derived from a pairwise shared secret
  - The pairwise shared secret is derived using a Pairwise Key Agreement Algorithm
    - Currently, this algorithm is based on Diffie-Hellman (e.g., ECDH-SS + HKDF-256)
    - No equivalent drop-in replacement; no standardized quantum-resistant DH/NIKE

[1] https://datatracker.ietf.org/doc/draft-ietf-core-oscore-groupcomm/

Contribution

- **Make the pairwise mode quantum-resistant**

- **Quantum-resistant variant of the derivation of pairwise keys**
  - Based on a quantum-resistant Key Encapsulation Method (KEM), like ML-KEM (FIPS 203)
  - *draft-ietf-jose-pqc-kem* [2] is registering COSE Algorithms for ML-KEM

- **No change to the use of pairwise keys for message processing**

- **Take advantage of the Group Manager in the group**
  - Like it already collects and distributes the authentication credentials of group members …
  - It facilitates the exchange of the KEM public keys and KEM ciphertexts to establish secrets
  - Details are left to the specific realization of Group Manager

[2] https://datatracker.ietf.org/doc/draft-ietf-jose-pqc-kem/

Derivation of pairwise keys

- **Kept as-is**
  - HKDF, info, and L
  - Sender Key
  - Recipient Key
  - Sender Auth Cred
  - Recipient Auth Cred
  - Use of Pairwise Sender Key
  - Use of Pairwise Recipient Key

- **New when using a KEM**
  - Sender Shared Secret to compose IKM-Sender
  - Recipient Shared Secret to compose IKM-Recipient

```
Pairwise Sender Key    = HKDF(Sender Key, IKM-Sender, info, L)
Pairwise Recipient Key = HKDF(Recipient Key, IKM-Recipient, info, L)

with

IKM-Sender    = Sender Auth Cred | Recipient Auth Cred | Shared Secret
IKM-Recipient = Recipient Auth Cred | Sender Auth Cred | Shared Secret
```

Old construction using Diffie-Hellman

```
Pairwise Sender Key    = HKDF(Sender Key, IKM-Sender, info, L)
Pairwise Recipient Key = HKDF(Recipient Key, IKM-Recipient, info, L)

with

IKM-Sender    = Sender Auth Cred | Recipient Auth Cred |
                Sender Shared Secret
IKM-Recipient = Recipient Auth Cred | Sender Auth Cred |
                Recipient Shared Secret
```

New construction using a KEM

From an endpoint's perspective

- **Sender Shared Secret**
  - Computed using the KEM public key of the other endpoint X
  - Encapsulated in the KEM ciphertext intended to X

```
Pairwise Sender Key    = HKDF(Sender Key, IKM-Sender, info, L)
Pairwise Recipient Key = HKDF(Recipient Key, IKM-Recipient, info, L)

with

IKM-Sender    = Sender Auth Cred | Recipient Auth Cred |
                Sender Shared Secret
IKM-Recipient = Recipient Auth Cred | Sender Auth Cred |
                Recipient Shared Secret
```

- **Recipient Shared Secret**
  - Computed using the private key of this endpoint …
  - By decrypting the KEM ciphertext computed by the other endpoint X and intended to this endpoint

- **A KEM public key is used to compute only <u>one</u> KEM ciphertext, for <u>one</u> other endpoint**

- **KEM public keys and KEM ciphertexts are exchanged through the Group Manager**

Assistance from the Group Manager

- **Section 4 – Operations provided by the Group Manager at a high level**
  - Details are left to the particular realization of Group Manager
  - For example, the ACE-based Group Manager [3] can be extended as defined in [4]

- **Uploading a KEM public key, for any and exactly one other endpoint**
  - Already when joining the group, or later on at any time as a group member

- **Retrieving a KEM public key of X, by specifying the Sender ID of X**

- **Uploading a KEM ciphertext intended to X**
  - Together with the Sender ID of X and a hash-based identifier of the KEM public key of X that was used

- **Retrieving a KEM ciphertext, by specifying the Sender ID of X that computed it**
  - Retrieved together with the hash-based identifier of the own KEM public key that was used by X

[3] https://datatracker.ietf.org/doc/draft-ietf-ace-key-groupcomm-oscore/
[4] https://datatracker.ietf.org/doc/draft-tiloca-ace-oscore-gm-kem/

Next steps

- **Section 3.1 “Use of ML-KEM (FIPS 203)”**
  - More details/guidelines on the use of ML-KEM
  - Largely building on *draft-ietf-jose-pqc-kem*

- **Comments are welcome!**

Thank you!

Comments/questions?

https://gitlab.com/crimson84/draft-tiloca-core-group-oscore-kem